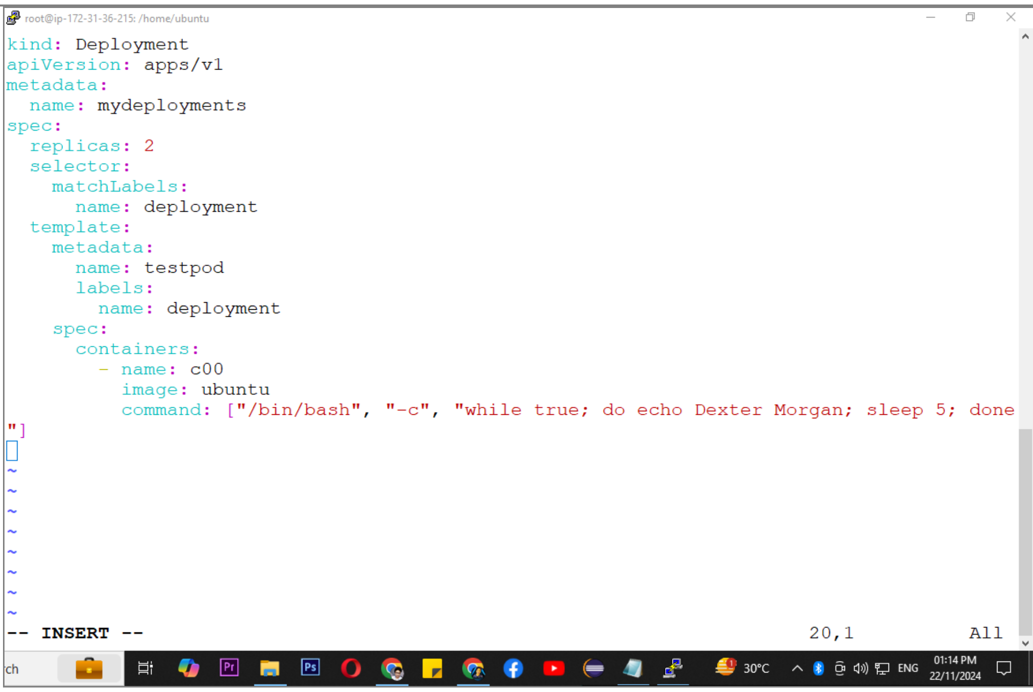
Deployment Object in Kubernetes (Rollout, Rollback, Versioning)

Rohit Surwade

Deployment manifest



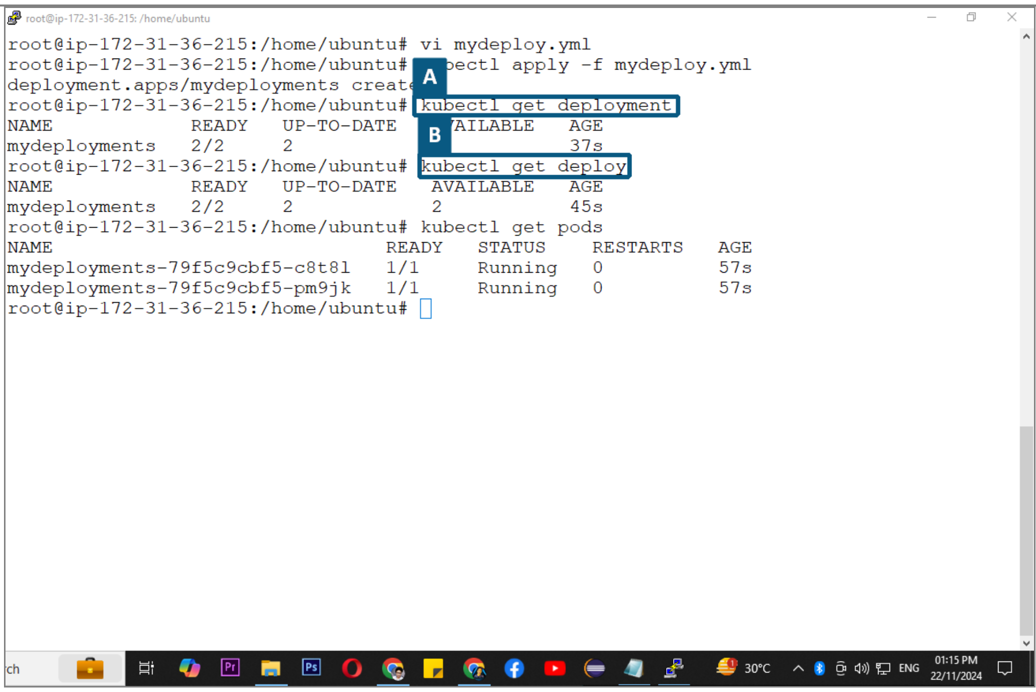
```
root@ip-172-31-36-215: /home/ubuntu
kind: Deployment
apiVersion: apps/v1
metadata:
  name: mydeployments
spec:
  replicas: 2
  selector:
    matchLabels:
      name: deployment
  template:
    metadata:
      name: testpod
      labels:
        name: deployment
    spec:
      containers:
        - name: c00
          image: ubuntu
          command: ["/bin/bash", "-c", "while true; do echo Dexter Morgan; sleep 5; done
"]
~
~
~
~
~
~
~
~
-- INSERT --                                                  20,1          All
```

1.

(This manifest defines a Kubernetes Deployment resource. A Deployment is a higher-level abstraction for managing pods in Kubernetes,
ensuring that the desired number of replicas of a pod are running and updated properly.mydeployments is the name of the deployment.
This is how you will reference this deployment in the cluster. selector: Defines how the deployment identifies which pods to manage.
A selector is used to match pods with specific labels. matchLabels: The deployment will look for pods with the label
name: deployment to manage. This is a way for the deployment to target the pods it is responsible for.
Deployment is a higher-level abstraction that provides a robust mechanism for managing applications,handling rolling updates and rollbacks, and ensuring availability.
ReplicaSet focuses solely on maintaining the desired number of pod replicas without the features of automatic updates and rollbacks.
Deployment: Manages the lifecycle of applications, including handling rolling updates and rollbacks when changes are made to the application.
Deployments use ReplicaSets internally to manage the pods.
ReplicaSet: Focuses purely on maintaining the number of replicas of a pod. It does not manage updates or rollbacks on its own.)

```
kind: Deployment
apiVersion: apps/v1
metadata:
name: mydeployments
spec:
replicas: 2
selector:
matchLabels:
name: deployment
template:
metadata:
name: testpod
labels:
name: deployment
spec:
containers:
- name: c00
image: ubuntu
command: ["/bin/bash", "-c", "while true; do echo Dexter Morgan; sleep 5;
done"]
```

How to list the deployment objects(i.e ReplicaSet)

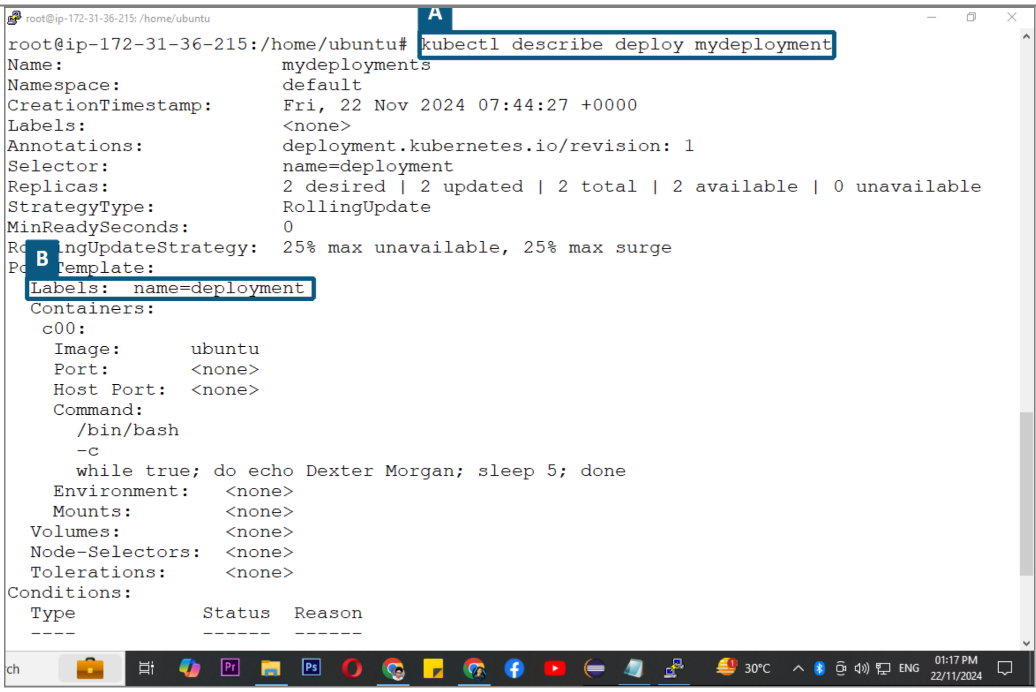


2.

commands:
kubectl get deploy
OR
kubectl get deployment

NOTE:
Deployment: Supports rolling updates and rollbacks. When you update the image or configuration of the container, the Deployment ensures zero downtime by gradually updating pods.
ReplicaSet: Does not manage updates. If you want to update the pods, you would need to manually update the ReplicaSet, which won't handle graceful updates.

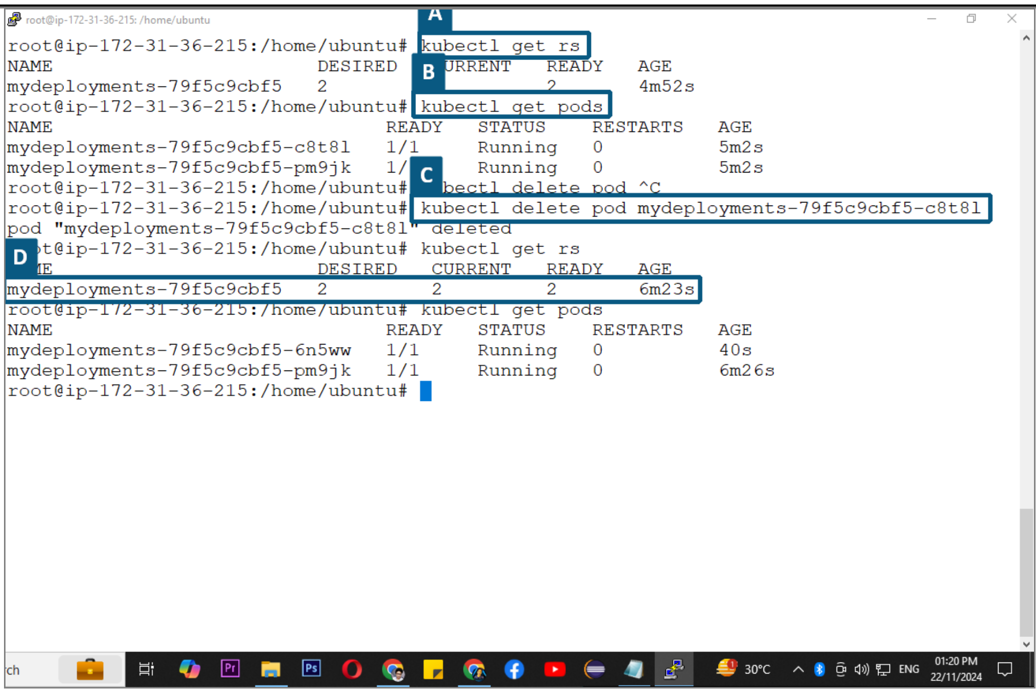
How to check deployment details



3.

kubectl describe deploy deployment-object-name
kubectl describe deploy mydeployment

As per our instructions in the Yaml manifest Deployment object has created 2 replicas



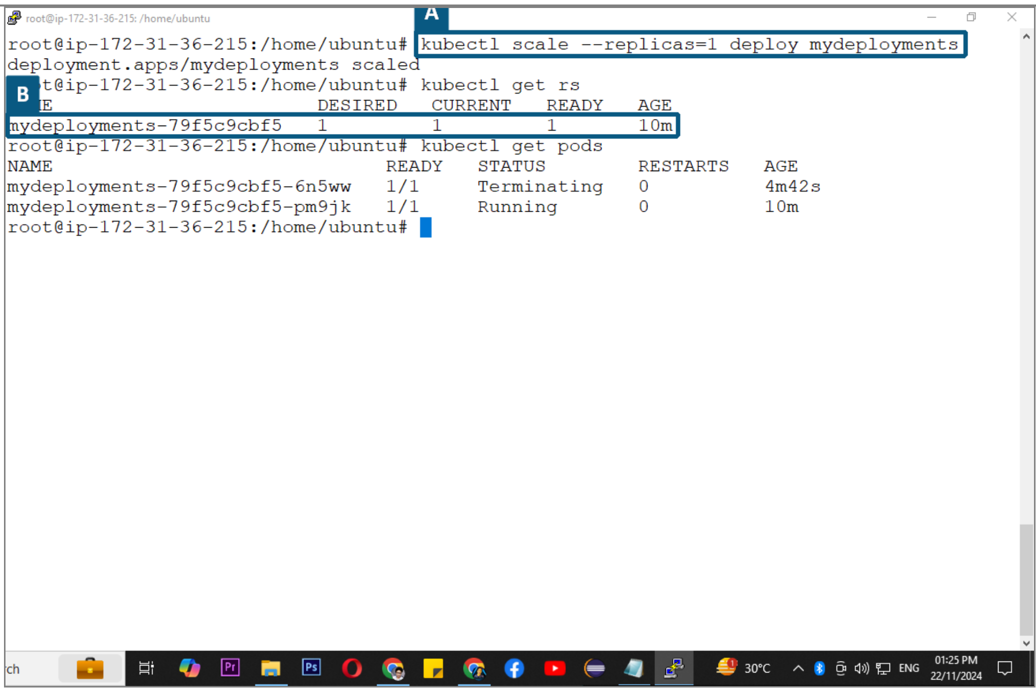
4.

commands:

1. kubectl get rs
2. kubectl get pods
3. kubectl delete pod pod-name.

NOTE:
ReplicaSet will maintain the desired state(i.e replica=2) Even if we delete a pod it'll immediately create a new one

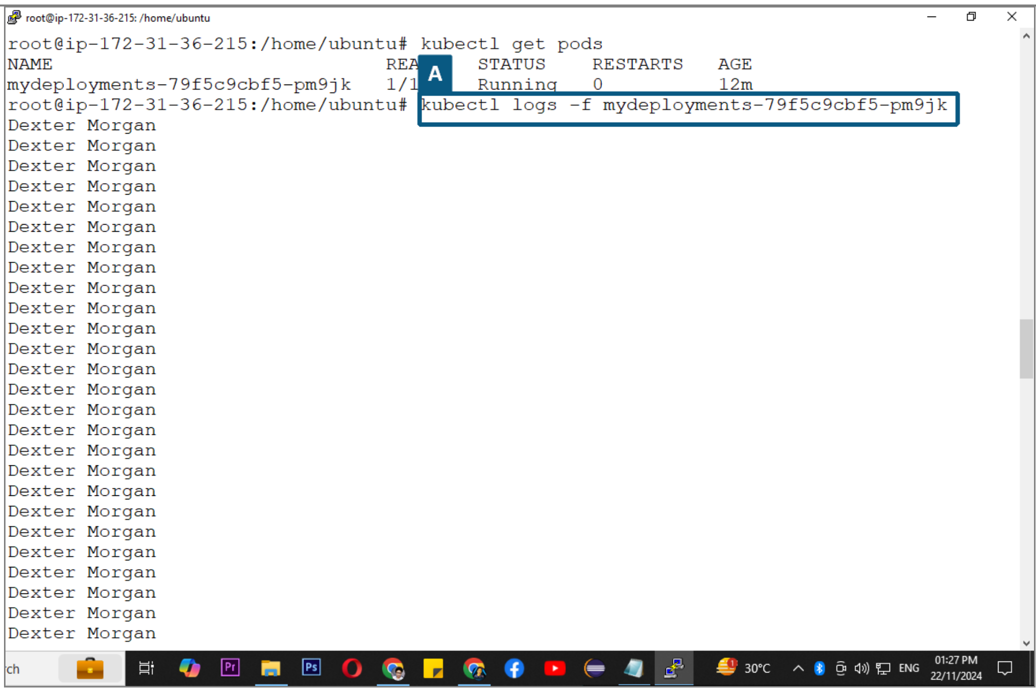
How to scale down Deployment objects(Replicas)



5.

kubectl scale replicas=*Desired number* deploy deployment-object-name
kubectl scale replicas=1 deploy mydeployments

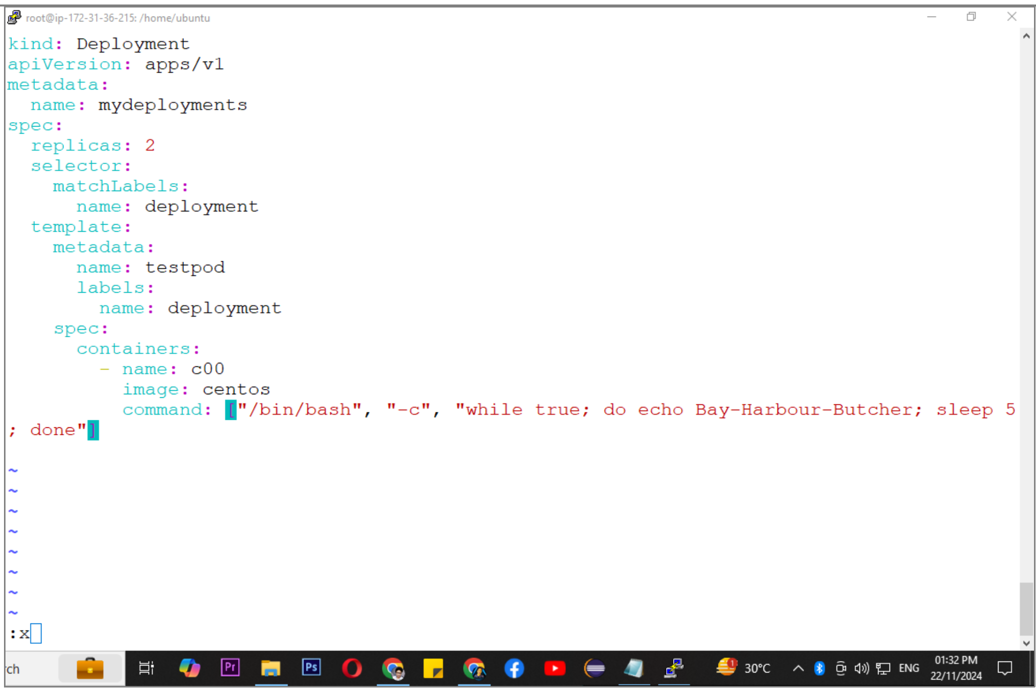
How to check logs of the pods



6.

kubectl logs -f pod-name

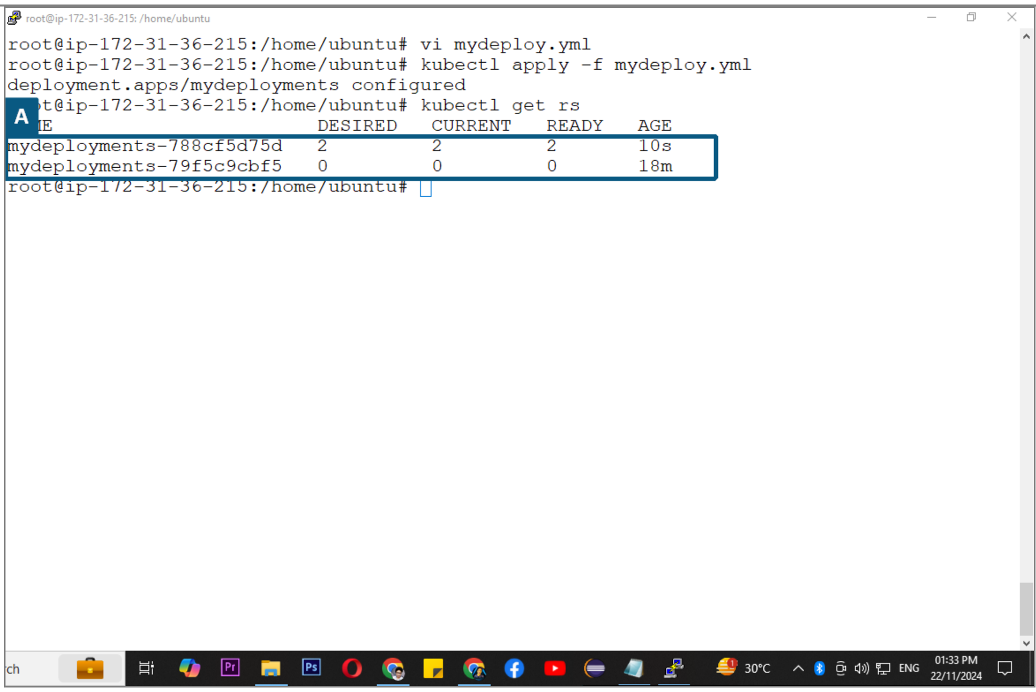
Making changes to the Deployment Manifest file and Applying them



7.

NOTE:
We changed the os from “ubuntu” to “centos” and
echo command from “Dexter Morgan” to “Bay-Harbour-Butcher”

Changes occured part-1

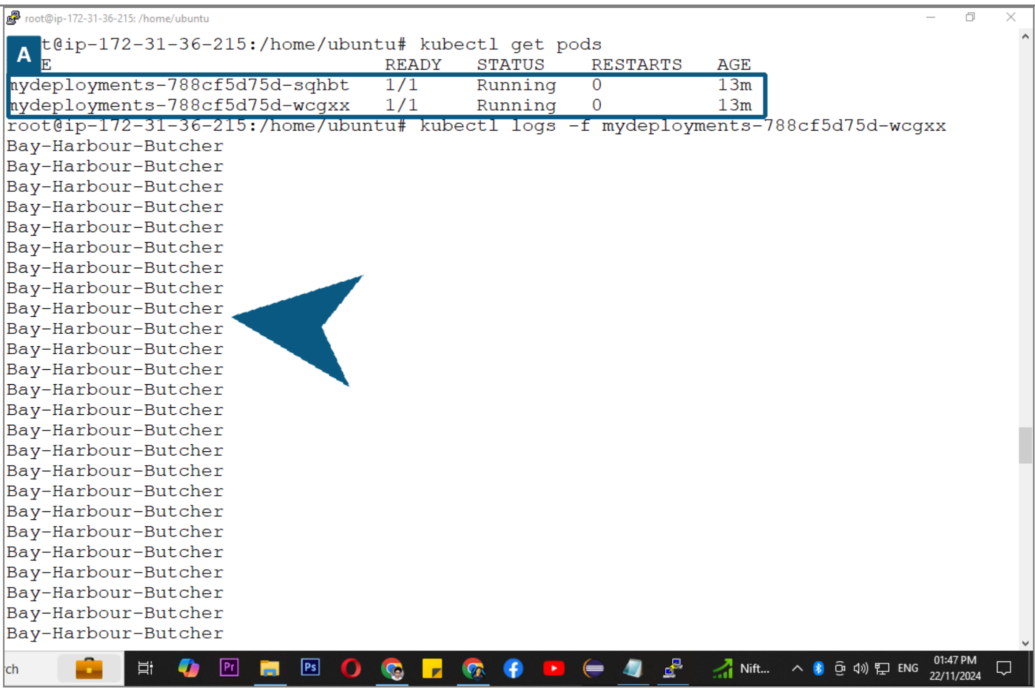


8.

NOTE:

1. Previous replicas have been freezed and new Replicas are created
2. We can call it a new version

Changes occured part-2

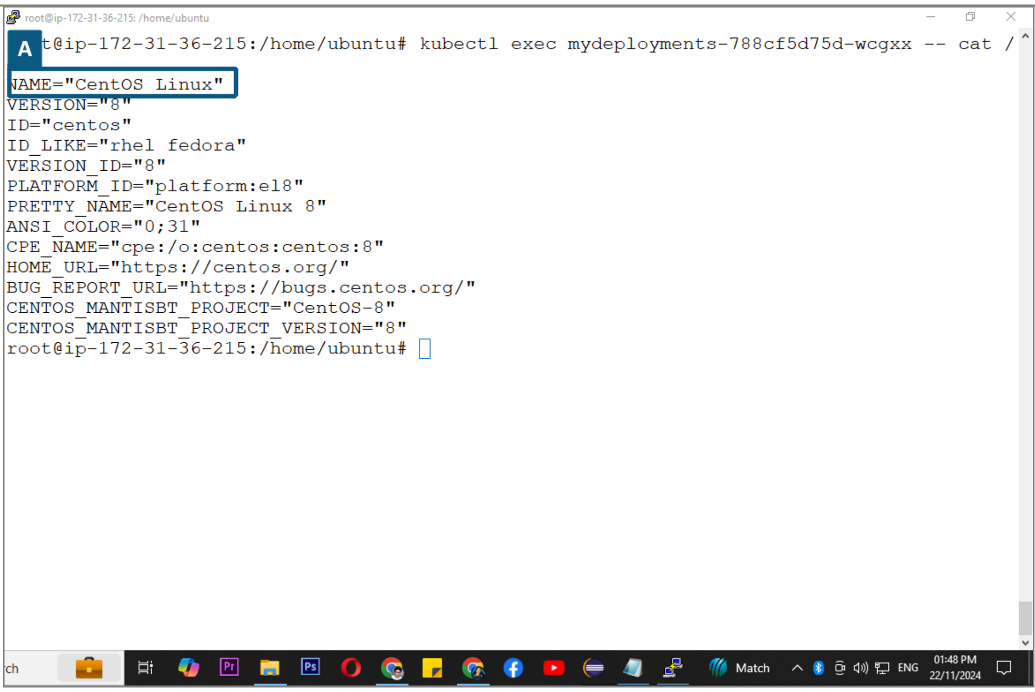


9.

NOTE:

1. Newly created replica(pod) names will begin with the deployment-object-name in our case “my deployments” .
2. echo command has been changed as per our instructions.

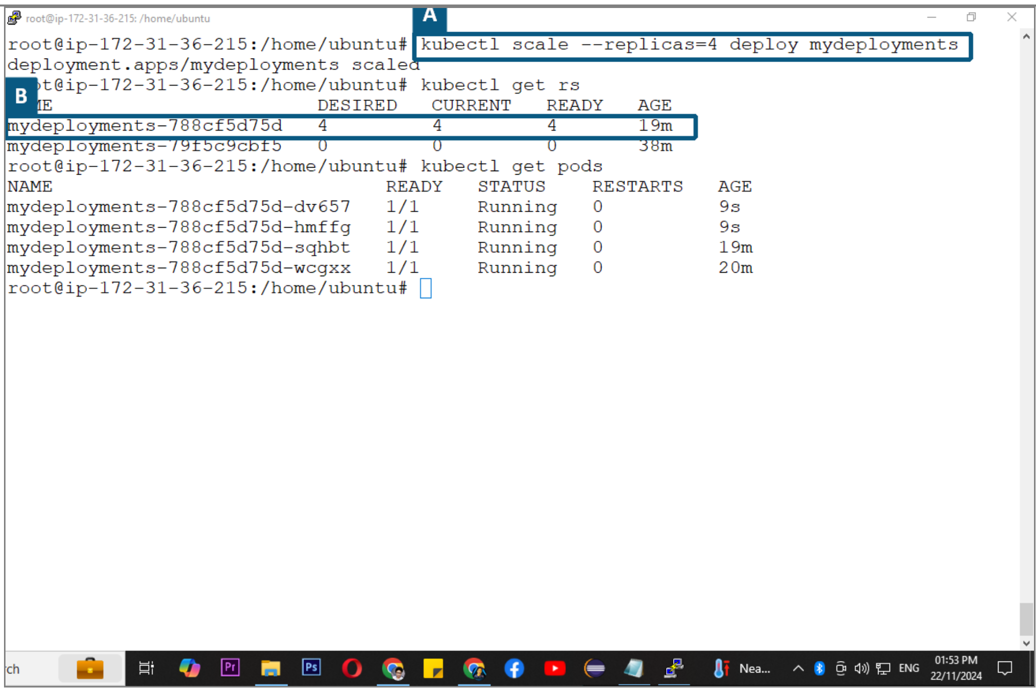
Changes occured part-3



10.

The os has also been also updated from "ubuntu“ to ”centos“

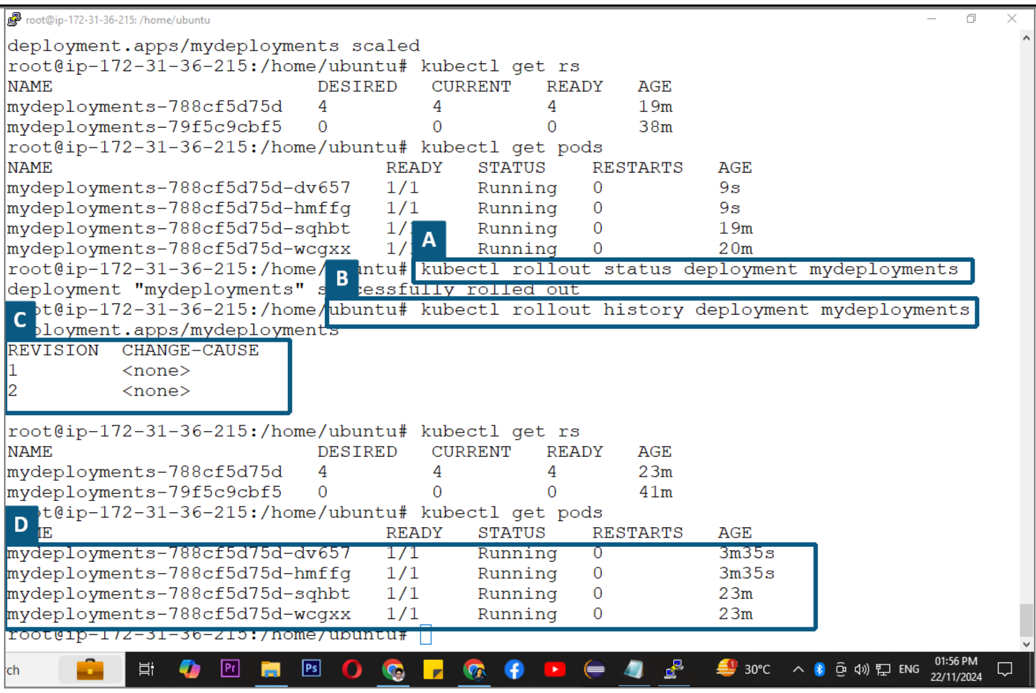
How to scale up Deployment objects(Replicas)



11.

kubectl scale replicas=*Desired number* deploy deployment-object-name
kubectl scale replicas=4 deploy mydeployments

How to rollout to previous version and check rollout history



12.

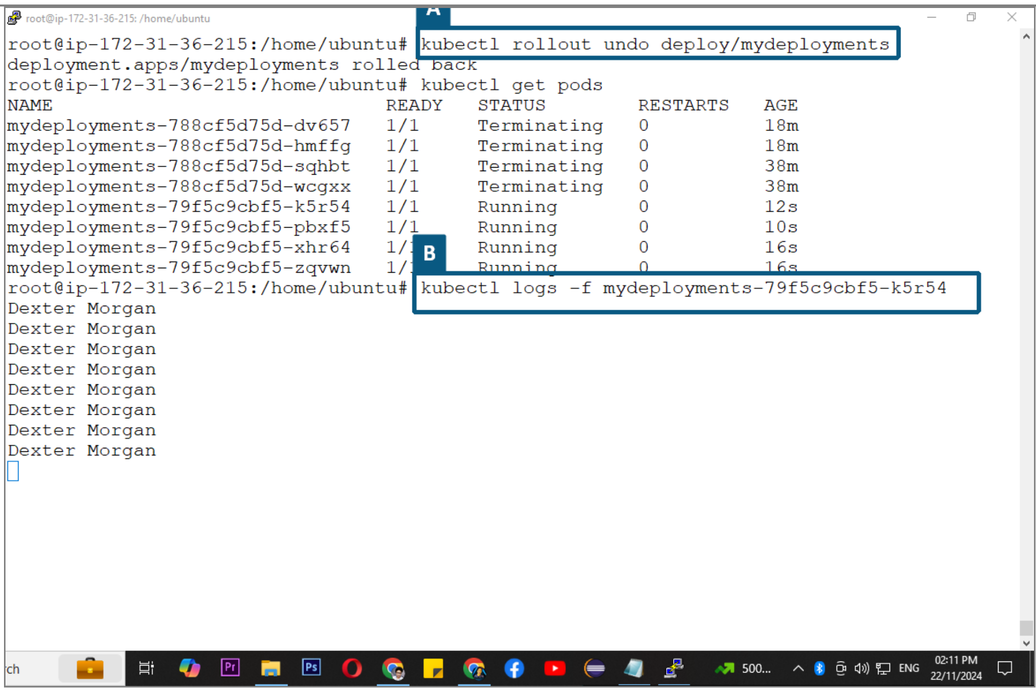
commands:

1. kubectl rollout status deployment deployment-object-name
2. kubectl rollout history deployment deployment-object-name

NOTE:

- Two changes(versions) are reflecting as we have made changes twice.
- The replicas(pod) numbers will remain the same(i.e if we were using 4 replicas in the current operation even if we rollback the replica number will be the same even though we were working on 2 replicas in the previous version)
- However this command will not apply the rollout changes(previous version).

How to apply the rollout changes



13.

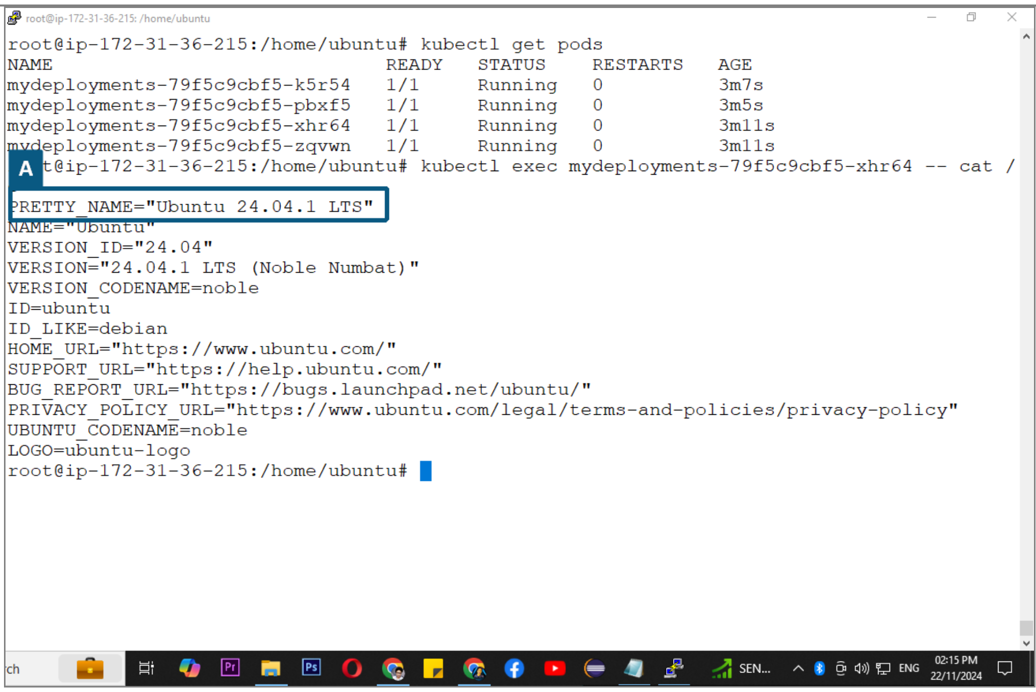
commands:

1. kubectl rollout undo deploy/deployment-object-name
2. kubectl logs -f pod-name

NOTE:

- The rollout changes have been applied previous pods are being terminated and new ones are being created with the previous code and configurations(such as os and other softwares).
- The echo command has been changes(rolled back to previous version).

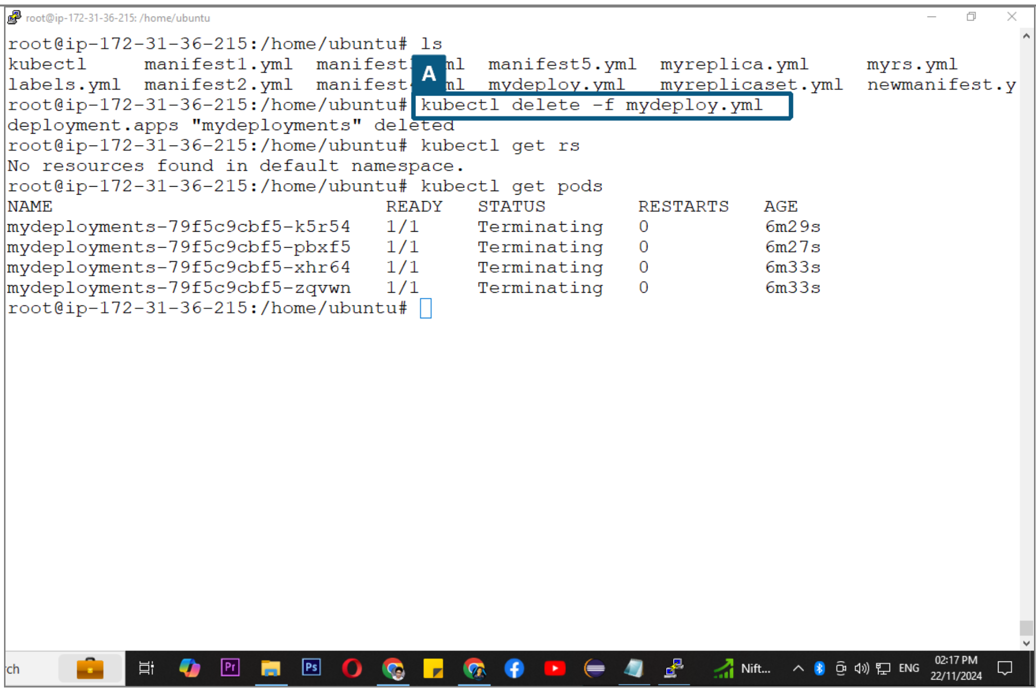
Rollback changes



14.

As we can see the os has also been rolled out to the previous version i.e “ubuntu”

How to DELETE the Deployment object



15.

kubectl delete -f mydeploy.yml